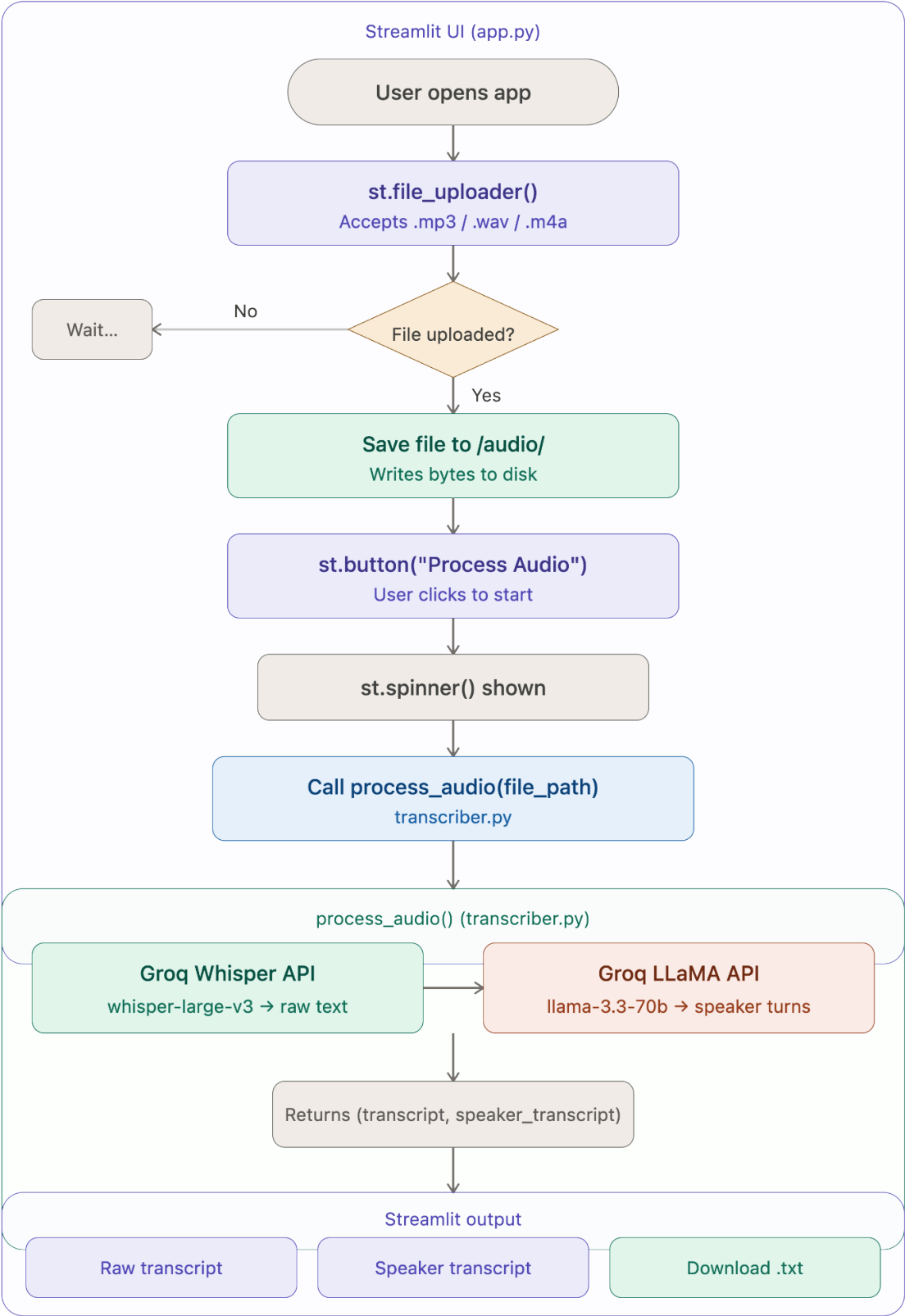


Code Flow - Voice Transcriber



`app.py` handles everything the user sees and touches, and `transcriber.py` does the actual heavy lifting with AI. The diagram is organized into three colored zones reflecting this separation.

Zone 1 - The Streamlit UI (Purple)

This is where the user's journey begins. When someone opens the app, the first thing they hit is `st.file_uploader()`, which accepts `.mp3`, `.wav`, or `.m4a` files.

There's a **decision gate** (the diamond shape) right after this. If no file is uploaded, the app just sits and waits - nothing happens. Once a file comes in, the flow continues downward.

The uploaded file doesn't get processed in memory directly. Instead, it gets **written to disk** inside an `/audio/` folder. This is intentional - Groq's API needs a real file path, not an in-memory buffer.

After the file is saved, a "Process Audio" button appears. The app does nothing until the user explicitly clicks it. Once clicked, a `st.spinner()` activates, giving visual feedback that something is running in the background.

The Bridge — `process_audio(file_path)`

This is the handoff moment. The Streamlit layer calls `process_audio()` from `transcriber.py`, passing the saved file path. From here, control leaves the UI entirely and enters the processing layer.

Zone 2 — The Transcriber (Teal + Coral)

This is where the two AI calls happen, and they're **sequential, not parallel** - the output of the first feeds directly into the second.

Step 1 — Groq Whisper (Teal box): The audio file is opened and sent to Groq's Whisper large-v3 model. It comes back as a plain raw text transcript - no speaker info, no formatting, just a wall of words.

Step 2 — Groq LLaMA (Coral box): That raw transcript is injected into a carefully crafted prompt and sent to `llama-3.3-70b-versatile`. The LLM's job is to read the conversation and figure out who said what — reorganizing everything into labeled speaker turns (`Person 1:`, `Person 2:`). The `temperature=0` setting is important here; it keeps the output deterministic and consistent.

The function then returns **both** outputs — the raw transcript and the speaker-labeled version — as a tuple back to `app.py`.

Zone 3 — Streamlit Output (Purple again)

Back in the UI, the two return values get displayed in separate `st.text_area()` blocks side by side. The raw transcript lets users verify the transcription accuracy, while the speaker transcript shows the cleaned-up conversation. A **download button** at the end lets users save the speaker transcript as a `.txt` file locally.